

Appendix: Measurement Methodology

Reproduction, evidence scope, and the limits of the measurements

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

A1. Timing and provenance

The headline results use the course QEMU/KVM VM, Ubuntu 22.04, release CPython 3.10.12 and pyperformance 1.14.0. Each JSON contains 120 measured values (40 value-bearing worker runs, three values each); calibration is separate. Reported uncertainty is sample standard deviation across those values, not a confidence interval. Use the full-precision JSON means to calculate ratios, rather than rounded display values.

Comparison	Files in the results directory
Nbody stock / Python	baseline_nbody.json / optimized_nbody.json
Pyflate stock / Python	baseline_pyflate.json / optimized_pyflate.json
Nbody Python / native	fallback_nbody.json / native_nbody.json
Pyflate Python / native	fallback_pyflate.json / native_pyflate.json

The stock/optimized and fallback/native comparisons were separate experiments. Older stock/optimized JSONs lack backend metadata; the later fallback/native files explicitly record python and native. The pure-Python means agree closely across the two experiments.

Development evidence is labeled separately. Pyflate's ablation and cProfile tables come from Windows / CPython 3.12.6. Nbody's detailed unrolling sweep and pyflate's older 25× native kernel example come from WSL2 / CPython 3.10.21. Different machines, interpreters and estimators prevent using these absolute timings as VM results. Likewise, a paired median speedup need not equal the ratio of two independently reported minimum times. Bytecode counts collected with `sys.monitoring` also depend on the CPython version and counting workload; they are not portable instruction counts for the VM's interpreter.

Reproducing the comparisons

From the repository root, compare saved data without changing it:

```
python3 -m pyperf compare_to results/baseline_nbody.json \
    results/optimized_nbody.json
python3 -m pyperf compare_to results/fallback_pyflate.json \
    results/native_pyflate.json
```

For fresh native/Python comparisons, use the same interpreter with the wheels installed:

```
HWSW_BACKEND=python python3 benchmarks/bm_nbody/run_benchmark.py \
    --rigorous --inherit-env HWSW_BACKEND -o /tmp/nbody-python.json
HWSW_BACKEND=native python3 benchmarks/bm_nbody/run_benchmark.py \
    --rigorous --inherit-env HWSW_BACKEND -o /tmp/nbody-native.json
```

Use new output names for each experiment. Repeat with `bm_pyflate` for decompression. `pyperformance run` creates its own environment; direct benchmark execution avoids assuming that environment contains the extension wheels. Native mode fails when a wheel is unavailable. The runner scripts at the repository root provide the full stock/optimized workflow.

Correctness

Nbody comparisons check all state components and energy after the same initial conditions and 20,000 steps. `Exact ==` is the observed result for the shipped Python and native kernels; a tolerance pass alone does not establish equality, and numerical equality is not a general byte-representation test (for example, signed zeros). Pyflate checks the decoded bytes against `bz2.decompress`, the original MD5, the intermediate L-vector and the ending bit position.

A2. Matched Python/native CPU counters

The new counter tables use `report/measure_native_counters.py` and the raw files in `results/native_counters_20260907/`. Release CPython 3.10.12 executes the same benchmark source in explicit Python and native modes, on one fixed guest CPU. Each configuration has three runs per event pass; backend order alternates between rounds. Every run warms up before measurement. Nbody resets state afterwards and executes 64 iterations of energy, 20,000 steps and energy; pyflate executes 16 complete decompressions. Output checks follow the timer.

Counter scope. `perf stat --delay=-1 --control fifo:...` starts with counters disabled. The worker enables counting after imports, setup, warmup and garbage collection, waits for acknowledgement, then times the loop. It disables counting afterwards and waits for a second acknowledgement. Counts include the small control-boundary overhead but exclude setup and output validation. The internal elapsed timer covers the loop only. Both core and generic cache events request user mode (:u); separate event passes are separate executions.

Pass	Events and validation
Core	Cycles, instructions, branches, branch misses: retained; reported running time 100%.
Cache	Generic cache references/misses: retained; reported running time 100%.
Excluded	L1 data-load counts were zero, despite reported 100% running time. Both L1 events are excluded; no L1 miss rate is inferred.

The summarizer checks source SHA-256 hashes, requested backends, output SHA-256 equality across all 24 runs, nonzero denominators and counter running time. A separate dispatch check (`report/check_pyflate_backend.py`) observed zero native calls for Python and one per block for native/auto, with identical MD5 and output length. The measured extension predates the subsequent source-only Rust module refactor; these timings are not a measurement of the rebuilt refactor.

Reported counts are normalized per completed benchmark iteration. Each displayed value is the median of three observations, including rates calculated **within each run**. Thus a ratio of displayed median counts may differ slightly from the displayed median rate. `summary.json` also retains the individual values and minimum/maximum. Three runs support a descriptive comparison, not a formal significance claim; the rigorous 120-value timing experiments remain the headline evidence.

Interpretation. IPC is retired instructions per counted cycle, not work completed per cycle. Removing interpreter instructions can lower IPC while improving runtime substantially. Generic cache events do not represent all memory accesses or identify stall cycles, a specific cache level, or DRAM latency. Nbody's very small miss counts are particularly sensitive to boundary overhead and run-to-run variation. Pyflate's larger remaining counts cannot be assigned to BWT without an isolated phase measurement.

To regenerate the validated summaries from the preserved capture:

```
python3 report/summarize_counters.py
```

For a new capture on Linux, choose a fresh output directory:

```
python3 report/measure_native_counters.py --repo /path/to/repo \
  --out /tmp/new-native-counters
```

A3. Earlier phase counters: evidence and limits

`dev/pyflate/phase_cpi.py` repeats one phase after preparing its input. The internal wall timer covers the repeated phase; `perf stat` wraps the **entire process**, including imports and input preparation. The values below reproduce `results/pyflate_phase_cpi.txt`; wall times are divided by repeat count and rounded. These are phase timing and aggregate-counter measurements, not a CPI stack decomposing execution into stall causes.

Phase	Repeats	ms/iter	IPC	Cache misses / refs
Native decode	300	3.304	1.49	1.284%
Whole inverse BWT	20	137.720	2.92	2.601%
Chase loop	30	47.820	2.41	1.831%
RLE4	100	23.836	2.89	1.623%

IPC uses instructions:u / cycles:u; the generic cache events were collected without that user-only modifier. Ratios are process-wide. Cache events do not identify which access missed, the latency it incurred, or how much useful work overlapped it. Therefore we do not infer an exact DRAM-access interval, or assign all cycles per traversal step to interpreter overhead.

The chase loop performs `end = T[end]`, reads `L[end]`, writes an output byte and runs Python loop machinery. Its setup builds `T` outside the wall timer but inside the counter interval. The `L`-vector contains 336,184 bytes; `T` is a Python list of indices, whose list slots and integer objects add memory. The final 399,360-byte output size is not the traversal table size. Whole BWT includes substantial table-construction work as well as the dependent traversal.

PMU and profiling configuration

- **Counting:** the VM experiments observed usable cycle/instruction counts and zeros when too many hardware events were requested together. Scripts split events into passes of at most four. Check actual counts and enabled/running time rather than assuming multiplexing.
- **Sampling:** tested `cycles -F` invocations produced zero samples, whereas fixed-period `cycles -c` and software `cpu-clock` sampling worked. The experiment demonstrates a working configuration; it does not establish the kernel's internal cause or a universal KVM limitation. Core and reference cycles are distinct events even when counts are close.
- **Call chains:** `--call-graph dwarf,16384` produced usable debug-build stacks where frame-pointer unwinding failed. Debug-build profiles describe that build's costs and are not used as release timings. Python-frame sampling uses `py-spy` separately.
- **Sample rate:** check `perf_event_max_sample_rate` after system tuning. A rate of 1 Hz was observed in the tuned setup and suppresses useful profiles. Do not assume it persists after a reboot or is the default on another machine.

Example counter command (whole-process scope):

```
HWSW_BACKEND=native perf stat \
  -e cycles:u,instructions:u,cache-references,cache-misses -- \
  python3 dev/pyflate/phase_cpi.py --phase chase --repeat 30
```

To establish a native BWT memory bottleneck, measure a native implementation, isolate or subtract setup with a justified method, and sweep working-set size. The current counters are insufficient for a measured SRAM-versus-DRAM accelerator comparison.

A4. Full profiles and review checks

The reports show full original flame graphs beside enlarged context crops. `report/make_figures.py` copies every recorded frame rectangle at its original coordinates; it does not merge functions, filter startup, cap stack depth or renormalize widths. Numbered outlines mark the same frame in the overview and its detail crop. Selected-frame titles, coordinates, sample weights and source filenames are recorded in `report/fig/profile_counts.json`.

Stock nbody uses the C-frame debug profile to expose interpreter operations. Optimized nbody and both pyflate figures use Python-frame py-spy profiles. The tall C graph needs a small overview to retain its complete context; the enlarged regions carry readable local detail. Original SVGs in `results/` remain available for arbitrary zoom and inspection.

Widths show **inclusive** samples for a call path, relative to the original whole profile. The C graph's weights sum sampled event periods, not a count of individual interrupts. A function may appear at several locations; the highlighted occurrence is not its aggregate self time. Children overlap their parents, so widths on different rows must not be added. Independent graphs are independently normalized: a narrower-looking stack is not evidence of an absolute speedup. Python-frame views have only a few hundred samples and are used to locate work, not resolve small percentage differences.

Earlier flame-graph trimming

Older trimmed py-spy views exclude startup samples; the original `*_full.svg` files preserve them. The C-frame figures in `results/flame_*.svg` also elide leading context and cap depth. Recorded caps are depth 9 / 4.5% affected samples for stock nbody and depth 36 / 4.4% for stock pyflate. Capping loses leaf detail. Removing samples can change the normalization, and merging frames can change individual box widths, even when surviving sample counts are preserved.

The C-level profiles remain available as supporting evidence: `results/flame_nbody_stock_full.svg`, `flame_nbody_opt_full.svg`, `flame_pyflate_stock_full.svg` and `flame_pyflate_opt_full.svg`. Flat self-time output is in the corresponding `perf_report_*.txt` files. Timings and headline speedups come from JSON measurements, not from graphical widths.

Review spot checks, 2026-09-07

Short checks were rerun on the course VM using the repository's existing scripts. SHA-256 hashes matched the local copies for both shipped benchmark files and both `rs_check.py` scripts. The following timings are **minimums of three interleaved rounds**, not new rigorous benchmark results:

Comparison	Python	Native / hybrid
Nbody stock integration	231.552 ms	9.458 ms
Pyflate T3 symbol loop	115.873 ms	3.201 ms
Pyflate T3 complete decode	278.244 ms	161.072 ms

Nbody's shipped Python kernel and its Rust kernel matched stock state and energy exactly after 20,000 steps. Pyflate's Python and hybrid outputs matched bz2; the L-vector and ending bit position matched too. These checks corroborate functionality and approximate performance, but do not replace the saved 120-value comparisons.

Fresh VM rerun

On 2026-09-07 we reran one rigorous pyperv job per backend pair on the course VM, using explicit `HWSW_BACKEND` settings and new temporary result files. The benchmark source SHA-256 values matched this checkout (nbody 226c5b9f...a47b5, pyflate 76c2361c...e4c494). The VM working tree also contained unrelated existing edits; no files were synchronized or replaced.

Fresh VM run	Python	Native	Ratio
Nbody	142 ms	9.48 ms	15.0×
Pyflate	286 ms	172 ms	1.66×

The fresh values are consistent with the saved 120-value distributions but are not merged into them: one rigorous job is useful as a revalidation, while the saved distributions remain the report's headline estimates and uncertainty values.

References: Python [profile semantics version-specific bytecode repository](#) and [raw data](#).